ICML 2024: Most Spiking Neurons Don't Need to Remember Time

Peking University, the Institute of Automation (CAS), and collaborators introduce T-RevSNN, cutting SNN training memory to O(L) and inference cost to O(1) while setting the best accuracy among CNN-based spiking networks on ImageNet

Spiking Neural Networks (SNNs) have long carried a promise: brain-inspired, low-power computation. Instead of moving continuous floating-point values around, they communicate with discrete spikes, which maps naturally onto neuromorphic hardware. But to make an SNN actually work well, you usually have to simulate it over many timesteps, and that is exactly where the cost creeps in.

The pain shows up at both ends of the pipeline. During training, backpropagation has to store intermediate activations for every layer at every timestep, so memory scales as roughly O(L×T), where L is depth and T is the number of timesteps. At inference, the same image is fed in T times, so energy scales with T as well. The frustrating part is that existing training tricks tend to relieve only one end: they save training memory or they save inference energy, rarely both.

This ICML 2024 paper introduces T-RevSNN (Temporal Reversible SNN), and it starts from a mildly counterintuitive observation: in most of the network, the temporal gradients of spiking neurons barely matter. If that is true, why pay the full temporal cost for every neuron? The authors' answer is to switch off the temporal dynamics of most neurons, keep them on only at a few key positions, and make those surviving temporal connections reversible. The payoff is O(L) training memory and O(1) inference cost, with almost no loss in accuracy.

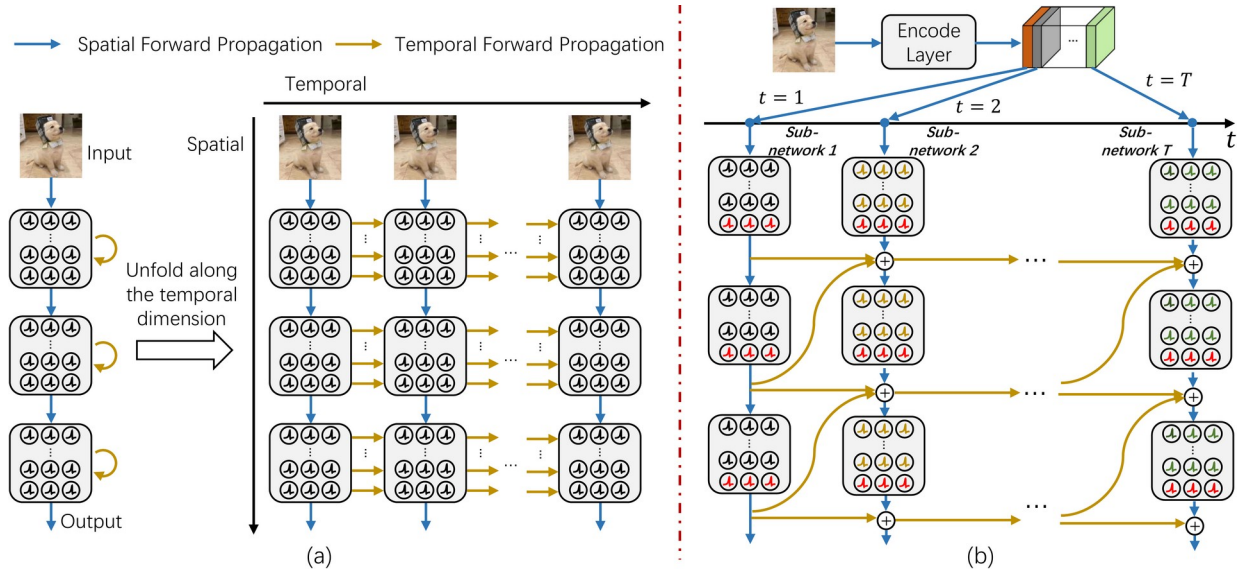

Figure 1. Temporal forward pass of a vanilla SNN versus T-RevSNN. (a) A vanilla SNN unfolds along time, re-feeding the image and reusing all parameters at every timestep, with every neuron carrying temporal dynamics; memory and inference cost are therefore O(L×T) and O(T). (b) T-RevSNN lets only a few key neurons (red)

pass temporal information; combined with a multi-level temporal-reversible design, memory drops to O(L). The image is encoded once, its features are split into T groups fed one per timestep, and the network is divided into T parameter-sharing sub-networks, so inference cost drops to O(1).

Paper: https://arxiv.org/abs/2405.16466

Code: https://github.com/BICLab/T-RevSNN

Why SNNs have been hard to scale

The memory bottleneck is not a small tax. Training a spiking ResNet-19 at 10 timesteps costs roughly 20 times the memory of a plain, non-spiking ResNet (a standard CNN) of similar size. That alone has kept SNNs out of the large-model regime: even when the algorithm is fine, the hardware simply can't hold it.

Prior work has chipped away at each end separately. Methods like OTTT and SLTT decouple training from the timestep, mainly easing memory pressure; other work fine-tunes to fewer inference steps, mainly saving inference energy. But each addresses only half of the dilemma, training memory or inference energy, never both at once. T-RevSNN asks whether both halves can be solved together.

The key observation: most temporal gradients don't matter

The authors begin by asking how much temporal gradients actually contribute. Using a spiking ResNet-18 on CIFAR-10, they run a controlled comparison: in case 1 they keep only the temporal gradients of the last layer of spiking neurons in each stage, and in case 2 they keep the temporal gradients of the remaining neurons instead. They then measure the cosine similarity between the resulting spatial gradients and those of the full baseline.

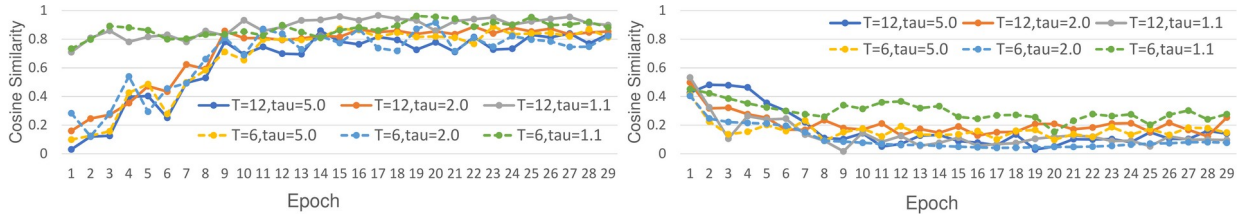

Figure 2. Cosine similarity of spatial gradients against the full baseline (left: case 1, right: case 2). Keeping only the last-layer-of-each-stage temporal gradients (case 1) leaves the spatial gradients almost unchanged (similarity near 1, left); keeping the other neurons' temporal gradients instead (case 2) yields low similarity (right). This says the temporal gradients at the final layer of each stage are the ones that matter, while those in the preceding layers can be dropped. The horizontal axis is training epoch; curves correspond to different timesteps T and decay τ.

The verdict is clean: only the last layer of each stage carries temporal gradients that count, while the earlier layers contribute almost nothing. So there is no need to maintain full temporal dynamics everywhere. Switch off the temporal dynamics of most neurons, keep them on only at the key positions, and accuracy is essentially preserved while both memory and compute are freed. That single observation is the seed for everything in T-RevSNN.

The method: turn temporal dynamics off, make what's left reversible

T-RevSNN's forward pass differs from a vanilla SNN in one fundamental way. Rather than re-feeding the same image at every timestep, it encodes the image just once, splits the encoded features into T groups, and hands one group to each timestep. The network itself is likewise divided into T sub-networks that share parameters and exchange temporal information only at the key, turned-on neurons. Because nothing is repeated T times at inference, inference cost collapses to O(1).

The surviving key connections follow a multi-level temporal-reversible rule: the membrane potential at one timestep can be exactly reconstructed from the next timestep's state and the incoming spikes. In other words, the forward pass is reversible, and that single property is what everything else in the design hinges on.

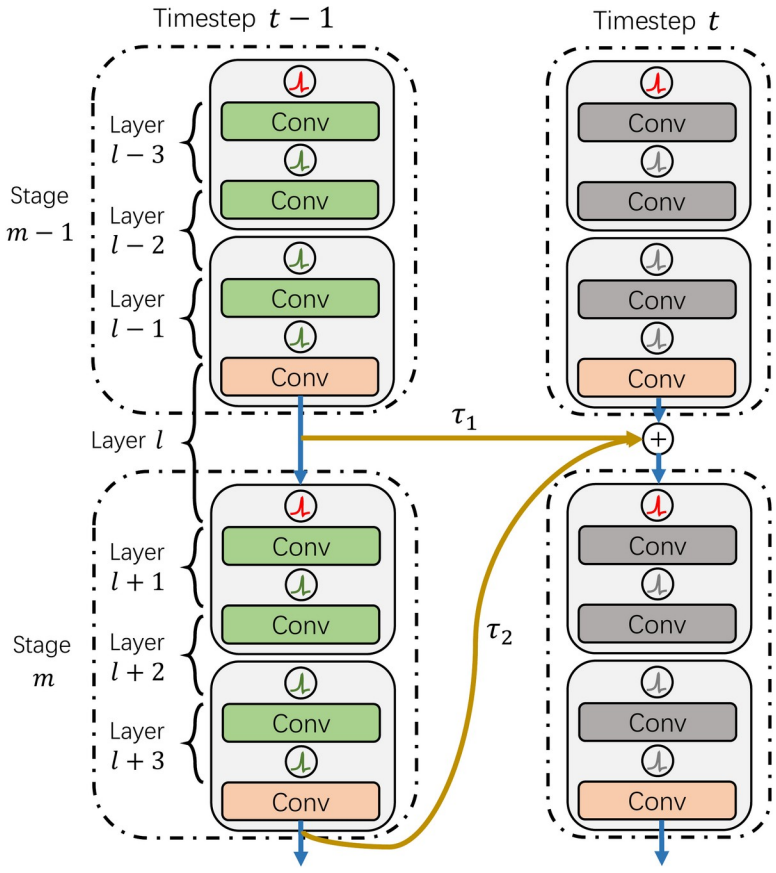

Figure 3. The temporal-reversible connection in T-RevSNN. Blue arrows are within-timestep spatial forward propagation; gold arrows carry temporal forward propagation across timesteps and across stages (τ_1, τ_2 are different decays). Only the key neurons at the last layer of each stage (red) pass and fuse information between adjacent timesteps, and that fusion is multi-level and reversible — the reversibility is what lets the backward pass avoid storing intermediate activations.

Reversibility buys one concrete thing: during the backward pass, the network no longer has to store activations for every layer and timestep. It only keeps the membrane potentials of the final timestep and recomputes the rest on the way back. That is what pulls training memory from O(L×T) down to O(L). To keep this sparse temporal interaction effective, the authors also fuse

features across levels, using up- and down-sampling to align features from different stages, so cross-stage temporal interaction is not lost.

Beyond the temporal changes, the authors redesign the basic SNN block in a ConvNeXt style: depth-wise and point-wise convolutions (DWConv/PWConv), no Batch Normalization, and a scaled residual connection with a learnable factor α. Together these keep the network training stably and converging quickly even under the sparser temporal interaction.

Seeing the training methods side by side

To make the difference from existing training schemes concrete, the authors draw the forward and backward passes of several methods together. The distinction lives in the backward pass, not the forward: STBP unfolds along both time and space; OTTT/SLTT unfold along space only; S-RevSNN unfolds along both but is reversible in space. T-RevSNN simply turns temporal dynamics off for most neurons, and for the few turned-on neurons it makes the backward pass reversible in time.

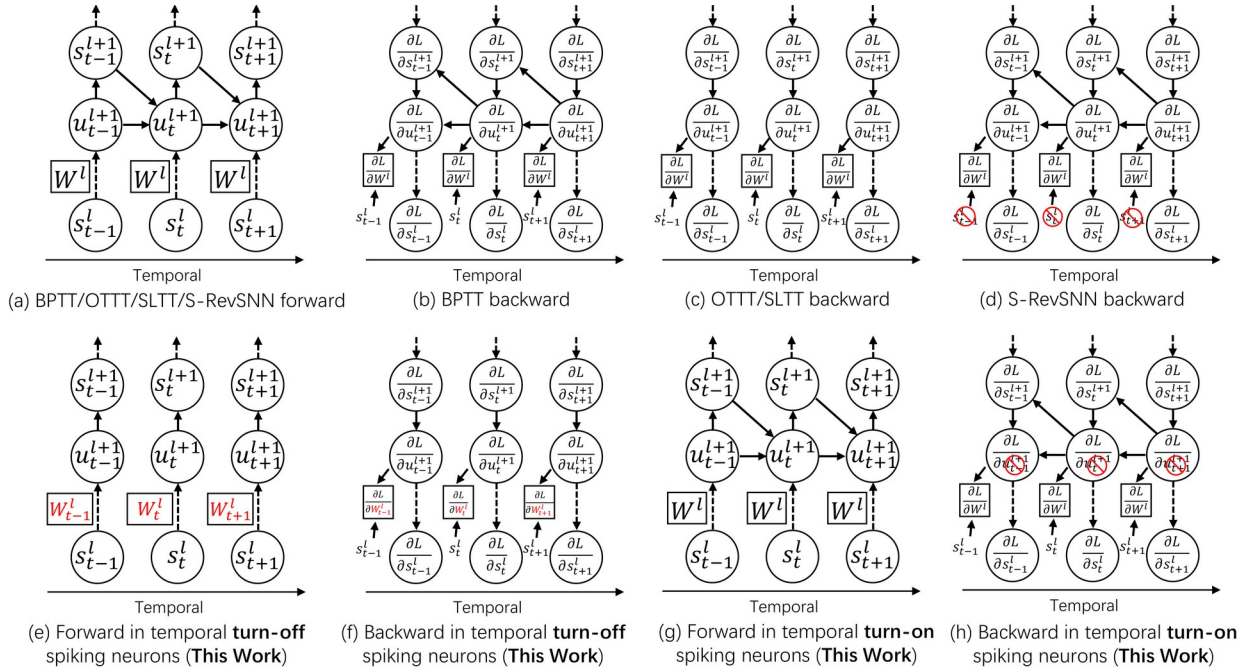

Figure 5. Forward and backward passes of the different training methods. (a) Existing methods don't change the SNN forward pass; (b) STBP's backward unfolds along both time and space; (c) OTTT/SLTT's backward unfolds along space only; (d) S-RevSNN's backward unfolds along both but is reversible in space; (e)(f) the forward and backward for this work's turned-off neurons; (g)(h) the forward and backward for the turned-on neurons, essentially matching (a)(b) except that the backward in (h) is reversible, so only the last timestep's membrane potentials must be stored.

Results

On static ImageNet-1K (224×224), T-RevSNN with a ResNet-18 backbone (512 channels in the last stage, about 29.8M parameters, T=4) reaches 73.2% top-1 accuracy while using just 85.7 MB/img of training memory and 2.8 mJ of inference energy. That is the best accuracy

among convolutional spiking ResNets, and it comes with the lowest training memory, the fastest training time, and the lowest inference energy in its class. All memory and time figures are measured on 6 NVIDIA A100-40GB GPUs under float16 mixed precision.

Against a leading spiking Transformer at comparable accuracy (the Spike-driven Transformer, 74.6%), T-RevSNN improves memory efficiency by 8.6 times, training speed by 2.0 times, and inference energy by 1.6 times, at essentially the same accuracy.

Table 1. Efficiency gains of T-RevSNN over a spiking Transformer of comparable accuracy (Spike-driven Transformer, 74.6% top-1), across three training/inference cost dimensions.

Dimension	Improvement
Training memory efficiency	8.6×
Training speed	2.0×
Inference energy	1.6×

The method is also validated on the neuromorphic datasets CIFAR10-DVS and DVS128 Gesture, confirming that T-RevSNN carries over to event-camera data as well.

Ablations: every design choice earns its place

The ablations show each component pulling its weight. The most direct one: apply temporal reversibility to a standard MS-ResNet-34, and training memory drops from 267.1 MB/img to 88.1 MB/img while per-epoch time falls from 11.2 to 7.4 minutes, at a cost of only about 1.6 accuracy points (68.3% → 66.7%).

Table 2. Ablation of applying temporal reversibility to an MS-ResNet-34 baseline. Memory and per-epoch time drop sharply, at a cost of roughly 1.6 accuracy points.

Setting	Accuracy	Memory (MB/img)	Time (min/epoch)
MS-ResNet-34 baseline	68.3%	267.1	11.2
+ temporal reversibility	66.7%	88.1	7.4

The other pieces contribute too: the multi-level temporal fusion between stages is worth about 1.2 accuracy points on its own (68.6% vs. 67.4%); the scaled residual connection speeds convergence noticeably, reaching 60% accuracy in 25 epochs instead of 32; and varying the timestep T trades accuracy against cost. The authors also confirm that temporal and spatial reversibility are orthogonal and can be stacked.

Takeaway and boundaries

The lasting message compresses to a single line: you don't need full temporal dynamics on every neuron. Because most temporal gradients in an SNN turn out to be unimportant, switching off the temporal dynamics of most neurons and making the few remaining connections reversible delivers O(L) training memory and O(1) inference cost, with almost no accuracy penalty.

Looking further out, T-RevSNN removes the memory and training-time bottleneck that has long stood in front of spiking networks, nudging them toward larger, more practical models. The gains rest on the observation that most temporal gradients don't matter, validated mainly on image classification and neuromorphic recognition; whether the same holds for tasks that lean more heavily on long-range temporal dependencies is a question for future work.